



UNIQ

Université Quisqueya

Faculté des sciences, de Génie et d'Architecture

Département d'Informatique

Document d'exigence

«OPERIX »

Identification Étudiant	
Code Etudiant :	Ar191008
Nom :	Arismé
Prénom(s) :	Djephane Asline Gabriel
Faculté :	FSGA
Option :	Génie informatique

Document d'exigence

Remerciements

À l'issue de ce projet, je tiens à exprimer mes sincères remerciements à toutes les personnes qui m'ont soutenue de près ou de loin pendant sa réalisation.

Je remercie tout d'abord mon professeur encadrant, **Monsieur Jean Andris ADAM**, pour ses conseils, sa patience et sa disponibilité. Ses remarques et orientations m'ont beaucoup aidée à mieux organiser mon travail et à avancer dans la bonne direction.

Je tiens aussi à remercier tous les enseignants de l'établissement pour les connaissances et les compétences qu'ils m'ont transmises tout au long de ma formation. Ces apprentissages ont été très utiles pour mener à bien ce projet.

Enfin, je remercie mon établissement pour le cadre de travail, les outils et les ressources mis à ma disposition, qui m'ont permis de réaliser ce projet dans de bonnes conditions.

Document d'exigence

Résumé

Ce document définit les exigences détaillées du projet OPERIX, une plateforme SaaS (Software as a Service) dédiée à la création et à la gestion de boutiques en ligne en Haïti. Il sert de cahier des charges fonctionnel et technique, décrivant avec précision les besoins auxquels le système doit répondre pour assurer une transition numérique efficace des commerçants locaux.

Le document expose d'abord le périmètre fonctionnel, identifiant les services essentiels tels que la gestion multi-tenante des boutiques, le catalogue de produits dynamique, le marché centralisé (Marketplace) et l'intégration des passerelles de paiement locales (MonCash, NatCash) et internationales. Les exigences non fonctionnelles y sont également traitées, mettant l'accent sur la sécurité (isolation des données), la performance (temps de réponse optimisé) et la réactivité de l'interface (responsive design).

Une section majeure est consacrée à la structure des données, détaillant les entités manipulées (Utilisateurs, Boutiques, Produits, Commandes, Paiements) selon le modèle implémenté avec l'ORM Prisma. L'architecture du système y est présentée, s'appuyant sur des technologies modernes telles que Next.js, MySQL et des services de stockage cloud, garantissant ainsi la robustesse et l'évolutivité de la solution.

Enfin, ce document précise les contraintes de développement, les règles de sécurité indispensables (chiffrement SSL, Row Level Security) et définit les rôles des différents acteurs du système. En somme, ce document d'exigence constitue le socle de référence pour le développement technique et la validation finale des livrables du projet OPERIX.

Document d'exigence

A. Identification

A.1 Identification du document

Nom du document : Document d'exigence
Projet : OPERIX
Version du projet : 1.1
Version du document : 1.1
Sécurité du document : Interne
Date de création : 03/02/2026
Par : Djephane Asline Gabriel ARISMÉ

A.2 Historique des changements

Qui ?	Quand ?	Quoi ?
Djephane A. G. ARISMÉ	03 février 2026	Création initiale du document
Djephane A. G. ARISMÉ	04 février 2026	Finalisation du document

Document d'exigence

Table of Contents

A.	Identification.....	3
A.1	Identification du document.....	3
A.2	Historique des changements	3
1.	Contexte et Justification.....	5
2.	Objectifs du projet.....	5
2.1.	Objectif général :	5
2.2.	Objectifs spécifiques :	5
3.	Périmètre du projet.....	6
3.1.	Périmètre fonctionnel.....	6
3.2.	Périmètre non fonctionnel.....	6
3.3.	Hors Périmètre.....	6
4.	Les données.....	7
5.	Architecture du système.....	8
5.1.	Composants du projet	8
5.2.	Flux typique dans le système.....	9
6.	Les Outils.....	9
6.1.	Outils de développement	9
6.2.	Outils de design et modélisation	10
7.	Sécurité du système	10
7.1.	Composants de sécurité.....	10
7.2.	Composants de communication.....	10
8.	Acteurs du système	10
9.	Contraintes.....	11
10.	Conclusion et Perspectives	12

Document d'exigence

1. Contexte et Justification

Aujourd'hui, la vente en ligne se développe partout, même en Haïti. Beaucoup de commerçants et de petites entreprises aimeraient vendre leurs produits sur internet pour toucher plus de clients. Cependant, créer son propre site web est souvent trop compliqué : cela demande beaucoup d'argent et des connaissances techniques pour gérer l'hébergement, la sécurité et les mises à jour.

À cause de ces obstacles, beaucoup abandonnent l'idée ou utilisent des solutions qui ne sont pas professionnelles. Il est donc devenu nécessaire de proposer une plateforme simple et fiable qui aide les gens à se lancer sans stress technique.

Le projet **OPERIX** répond à ce besoin. C'est une plateforme "tout-en-un" qui permet à chaque vendeur de créer sa propre boutique personnalisée en quelques clics. Le système s'occupe de toute la partie technique complexe, tandis que le vendeur se concentre uniquement sur ses produits et ses commandes.

La justification de ce projet est donc de simplifier la vie des commerçants. En centralisant la technique (sécurité, rapidité, stockage), OPERIX permet aux entrepreneurs de gagner du temps et de l'argent. Pour les acheteurs, c'est aussi un avantage car ils retrouvent plusieurs boutiques au même endroit, ce qui facilite leurs achats. OPERIX est donc une solution moderne et adaptée pour aider le commerce local à grandir sur internet.

2. Objectifs du projet

2.1.Objectif général :

Créer un site de vente en ligne sécurisé où chaque utilisateur peut ouvrir et gérer sa propre boutique facilement et de façon indépendante.

2.2.Objectifs spécifiques :

- ❖ Développer un tableau de bord intuitif permettant au vendeur de piloter son catalogue de produits, de suivre ses commandes et de configurer l'identité visuelle de sa boutique de manière autonome.
- ❖ Concevoir une interface de navigation fluide sur le marché centralisé, intégrant des outils de recherche avancés et un espace de gestion de profil pour le suivi des achats.
- ❖ Intégrer des passerelles de paiement sécurisées, incluant les solutions locales (MonCash, NatCash) et internationales, pour garantir la fiabilité des échanges commerciaux.
- ❖ Implémenter une structure de base de données multi-tenante utilisant un identifiant unique (tenant_id) afin d'assurer l'étanchéité et la confidentialité des données entre les différents vendeurs.

Document d'exigence

3. Périmètre du projet

3.1. Périmètre fonctionnel

Le projet inclura les fonctionnalités clés suivantes :

- ❖ **EF001 – Création de comptes** : Permet aux vendeurs (Tenants) et aux clients de créer un compte sécurisé pour accéder aux services.
- ❖ **EF002 – Gestion des boutiques** : Les vendeurs peuvent configurer leur boutique (nom, logo, URL unique) pour se différencier sur le marché.
- ❖ **EF003 – Gestion du catalogue** : Permet d'ajouter, modifier ou supprimer des produits (titre, prix, description, images et stock).
- ❖ **EF004 – Marché central (Marketplace)** : Les clients peuvent parcourir les produits de toutes les boutiques et filtrer par catégorie.
- ❖ **EF005 – Panier et Commande** : Permet au client de sélectionner des produits et de valider un panier d'achat.
- ❖ **EF006 – Paiement Mobile et Carte** : Intégration de MonCash, NatCash et des Cartes de débit/crédit pour valider les ventes.

3.2. Périmètre non fonctionnel

Les exigences de qualité retenues sont les suivantes :

- ❖ **ENF001 – Interface responsive** : L'application doit fonctionner sur mobiles et tablettes avec une navigation fluide.
- ❖ **ENF002 – Sécurité Multi-tenant** : Isolation totale des données ; un vendeur ne peut jamais voir les commandes d'une autre boutique.
- ❖ **ENF003 – Sauvegarde régulière** : La base de données doit être sauvegardée chaque jour pour éviter les pertes.
- ❖ **ENF004 – Performance** : Le temps de chargement d'une page produit ne doit pas dépasser 1.5 seconde.

3.3. Hors Périmètre

- ❖ **EF201 – Chat en direct** : La communication directe entre vendeur et client n'est pas incluse.
- ❖ **EF202 – Facturation automatique PDF** : La génération de facture PDF n'est pas prévue dans cette phase.

Document d'exigence

4. Les données

Dans cette partie, nous décrivons les entités essentielles manipulées dans le système **OPERIX**, conformément au modèle de données implémenté avec l'ORM Prisma.

❖ Un utilisateur possède les informations suivantes :

- **ID** : Identifiant unique auto-incrémenté.
- **Nom complet** : Nom et prénom de l'utilisateur.
- **Email** : Utilisé comme identifiant de connexion (unique).
- **Mot de passe** : Stocké sous forme de chaîne de caractères chiffrée.
- **Rôle** : Définit les permissions (ex: "acheteur" par défaut).
- **Token de réinitialisation** : Utilisé pour la récupération du mot de passe (resetToken).
- **Lien boutique** : Référence à la boutique possédée (pour les vendeurs uniquement).

❖ Une boutique possède les informations suivantes :

- **ID** : Identifiant unique.
- **Nom** : Le nom commercial de la boutique.
- **Sous-domaine** : URL unique pour l'accès direct (ex: ma-boutique.operix.com).
- **Activité** : Secteur de vente de la boutique.
- **Propriétaire (utilisateurId)** : Lien direct et unique avec l'utilisateur qui a créé la boutique.
- **Relations** : Liste des produits, des commandes et des paiements rattachés à cette boutique spécifique.

❖ Un produit contient les informations suivantes :

- **ID** : Référence unique de l'article.
- **Nom et Description** : Détails textuels du produit.
- **Prix** : Valeur décimale (précision 10, 2).
- **Stock** : Quantité entière disponible (0 par défaut).
- **Image URL** : Lien vers l'image hébergée du produit.
- **Dates** : Horodatage de création et de mise à jour automatique.
- **Boutique_ID** : Identifiant de la boutique parente (clé étrangère).

Document d'exigence

❖ Une commande contient les informations suivantes :

- **ID** : Identifiant unique de la transaction.
- **Total Prix** : Somme totale à payer.
- **Statut Commande** : État du flux (en_attente, paye, expédié, livré).
- **Infos Client** : Nom, téléphone et adresse de livraison (format texte).
- **Items (Snapshot)** : Liste des articles achetés stockée au format **JSON** (incluant le nom et le prix au moment de l'achat).
- **Boutique_ID** : Référence de la boutique concernée par la vente.

❖ Un paiement contient :

- **Méthode** : Type de paiement utilisé (MonCash, NatCash, Carte_crédit).
- **Statut** : État du transfert (en_attente, réussi, échoué).
- **Transaction ID** : Référence unique provenant du prestataire externe (Stripe, MonCash, etc.).
- **Montant** : Valeur monétaire de la transaction.
- **Lien Commande** : Référence à la commande payée.

5. Architecture du système

5.1. Composants du projet

- ❖ **Frontend (Next.js / React)** : Interface moderne permettant l'interaction avec le Marketplace et le Dashboard vendeur.
 - Module d'authentification
 - Tableau de bord vendeur (Gestion produits et ventes)
 - Interface client (Panier, Marché, Profil)
- ❖ **Backend (Next.js API Routes / Node.js)** : Gère la logique métier et l'isolation des données par boutique.
 - Service d'authentification (Clerk ou JWT)
 - Service de paiement (Connecteurs MonCash/NatCash)
 - Service de gestion des stocks
 - Service de notifications

Document d'exigence

- ❖ **Base de données (MySQL) :** Stockage structuré géré via l'ORM Prisma.
 - Tables Utilisateurs, Boutiques, Produits, Commandes.
- ❖ **Stockage de fichiers :** Utilisation d'un service (Cloudinary ou UploadThing) pour héberger les images des produits et les logos.

5.2. Flux typique dans le système

- Le client accède à une boutique via son URL unique.
- Il ajoute des produits au panier et clique sur "Payer".
- Le backend crée une commande "En attente" et redirige vers l'interface de paiement (ex: MonCash).
- Une fois le paiement validé, le fournisseur de paiement envoie un signal au backend.
- Le backend met à jour le stock, change le statut de la commande en "Payé" et informe le vendeur sur son Dashboard.

6. Les Outils

Pour le développement de la plateforme **OPERIX**, nous avons sélectionné des outils modernes permettant de garantir la performance du SaaS et la sécurité des paiements.

6.1. Outils de développement

- **Visual Studio Code :** Éditeur principal utilisé pour le codage en TypeScript et la gestion des fichiers du projet.
- **Next.js (React Framework) :** Utilisé pour le frontend et le backend (API Routes). Ce choix permet un rendu ultra-rapide des boutiques et un excellent référencement (SEO).
- **Prisma (ORM) :** Outil de liaison entre le code et la base de données. Il sécurise les requêtes et facilite la gestion du modèle multi-tenant.
- **MySQL :** Système de gestion de base de données relationnelle utilisé pour stocker de manière robuste les informations des vendeurs, des produits et des transactions.
- **Tailwind CSS :** Framework de design utilisé pour créer l'interface "Dark Premium" de manière rapide et responsive.

Document d'exigence

6.2.Outils de design et modélisation

- **Figma** : Utilisé pour concevoir les interfaces du Dashboard vendeur et du tunnel d'achat client avant le développement.
- **Draw.io / Lucidchart** : Outils retenus pour la réalisation des diagrammes UML (Cas d'utilisation, Séquence, et Schéma de base de données).

7. Sécurité du système

7.1.Composants de sécurité

- **Authentification sécurisée** : Utilisation de protocoles modernes pour protéger l'accès aux comptes vendeurs.
- **Chiffrement SSL/HTTPS** : Indispensable pour sécuriser les transactions par carte de débit/crédit et les données personnelles.
- **Isolation logicielle (Row Level Security)** : Système garantissant qu'un vendeur ne peut voir que les données liées à son propre tenant_id.

7.2.Composants de communication

- **Services de paiement (APIs)** : Connexion directe avec les serveurs de MonCash et NatCash pour la validation des paiements mobiles.
- **Passerelles Bancaires** : Intégration de modules sécurisés pour le traitement des cartes de crédit et de débit.
- **Notifications automatisées** : Envoi d'alertes par email ou SMS lors de la réception d'une nouvelle commande.

8. Acteurs du système

Acteur	Description
Vendeur (Tenant)	1) Crée sa boutique, 2) Gère son stock de produits, 3) Suit ses revenus.
Client (Acheteur)	1) Parcourt le marché, 2) Remplit son panier, 3) Paye ses achats en ligne.
Administrateur	1) Gère les inscriptions des vendeurs, 2) Supervise la sécurité globale.

Document d'exigence

9. Contraintes

Le projet **OPERIX** doit respecter plusieurs contraintes strictes pour assurer sa réussite.

❖ Contraintes techniques

- Utilisation obligatoire de Next.js pour l'ensemble de l'application.
- Base de données relationnelle sous MySQL.
- Gestion des données via l'ORM Prisma.
- Intégration fonctionnelle des paiements locaux (MonCash/NatCash).

❖ Contraintes de modélisation

- Respect des standards UML pour la documentation technique.

❖ Contraintes organisationnelles

- Travail avec une gestion des versions via Git/GitHub.
- Respect du calendrier de livraison (2 semaines).

❖ Contraintes de tests

- Tests de flux : Vérifier que l'argent est bien débité et le stock mis à jour.
- Tests de sécurité : S'assurer qu'un utilisateur non connecté ne peut pas accéder au Dashboard.
- Tests de performance : Validation du temps de réponse (inférieur à 2 secondes).

❖ Contraintes documentaires

Tous les livrables suivants doivent être fournis pour la validation finale :

- Document d'exigence
- Document d'analyse
- Document de conception
- Code source
- Démo
- Information du projet sur GitHub

10. Conclusion et Perspectives

Le déploiement de la plateforme **OPERIX** représente une avancée majeure pour les commerçants locaux, car il apporte une solution concrète aux difficultés de transition vers le commerce en ligne. En centralisant la gestion des produits, des ventes et des paiements, ce projet permet aux vendeurs de professionnaliser leur activité tout en garantissant une traçabilité totale des revenus. Cette première version d'OPERIX se concentre sur les fonctionnalités essentielles de vente et de gestion de Dashboard, offrant une base solide et sécurisée pour le commerce de proximité.

Dans une prochaine phase, la suite logique du projet sera d'étendre les capacités du système en intégrant des outils plus avancés. On pourra notamment ajouter un module de gestion des livraisons en temps réel, ainsi qu'un système de messagerie directe entre vendeurs et acheteurs. Il faudra également envisager l'intégration de rapports analytiques plus poussés (graphiques de croissance, prévisions de stock) pour aider les propriétaires de boutiques dans leur stratégie commerciale. Enfin, l'ouverture d'une application mobile dédiée permettrait d'offrir une expérience utilisateur encore plus fluide et accessible partout.